

Taller 3: Vectorización, Numpy y Exploración con Pandas

Docente: Dudbil Olvasada Pabon Riaño

1. Objetivo del Taller

Demostrar la superioridad en rendimiento de la vectorización matemática frente a la iteración clásica de Python. El estudiante importará y explorará datos tubulares utilizando Pandas y ejecutará operaciones masivas a nivel de CPU usando Numpy.

2. Instrucciones Generales

Desarrolle los siguientes retos en un Jupyter Notebook (Google Colab). En este taller **SÍ** está permitido y es obligatorio el uso de **numpy** y **pandas**. Documente su proceso utilizando celdas de Markdown. Al finalizar, descargue el notebook o suba el enlace a su repositorio.

Misión Analítica: IoT y Big Data

Una red de sensores IoT instalada en tractores agrícolas envía miles de mediciones de temperatura y humedad por minuto. El código actual en Python puro está colapsando el servidor por su lentitud. Su misión es refactorizar el procesamiento a Numpy y Pandas.

Reto 1: La Muerte del Bucle for (30 %)

Para simular el volumen de datos, cree una lista en Python con 1 millón de registros aleatorios de temperatura (entre 10.0 y 40.0 grados Celsius):

```
import random
temp_celsius = [random.uniform(10.0, 40.0) for _ in range(1000000)]
```

1. **El método lento:** Escriba una función clásica con un bucle **for** que convierta todas las temperaturas de Celsius a Fahrenheit ($F = C \times 1,8 + 32$). Mida el tiempo de ejecución usando el módulo **time**.
2. **La vía Numpy:** Convierta la lista a un arreglo de Numpy (**np.array**). Ejecute la misma operación matemática directamente sobre el arreglo completo (vectorización). Mida el tiempo de ejecución.
3. **Análisis Crítico:** Calcule cuántas veces más rápido es Numpy en comparación con el bucle tradicional y explique **por qué** ocurre este fenómeno a nivel de arquitectura (pista: C vs Python y memoria contigua).

Reto 2: Álgebra Lineal Descriptiva (20 %)

Usted tiene una matriz de 3×3 que representa las ventas (en miles de USD) de 3 sucursales durante 3 días.

$$\text{Ventas} = \begin{bmatrix} 12 & 15 & 10 \\ 20 & 18 & 22 \\ 8 & 9 & 11 \end{bmatrix}$$

1. Utilice `numpy` para construir la matriz bidimensional (`np.array`).
2. Utilice funciones vectorizadas para responder:
 - ¿Cuál fue la venta máxima registrada en toda la matriz?
 - ¿Cuál fue el promedio de ventas por cada **día** (columnas) (usando el argumento `axis`)?
 - ¿Cuál fue el total de ventas por **sucursal** (filas)?

Reto 3: Exploración Sensorial con Pandas (30 %)

Para este reto, debe investigar cómo generar un DataFrame de juguete usando Pandas.

1. Cree un `pd.DataFrame` de 5 filas y 4 columnas. Las columnas deben ser: `['id_sensor', 'fecha', 'temperatura', 'estado']`. Llene los datos con valores inventados realistas (el estado puede ser `.'Activo.'` o `.'Inactivo.'`).
2. Utilice los métodos `.head()`, `.info()` y `.describe()` sobre el DataFrame e interprete lo que cada uno arroja en una celda de Markdown.
3. Ejecute la instrucción `df['temperatura'].mean()` y explique qué tipo de objeto (clase) retorna extraer una sola columna de un DataFrame (Series vs DataFrame).

Reto 4: Filtrado Booleano Inteligente (20 %)

Del DataFrame creado en el Reto 3, el equipo de mantenimiento necesita una lista de los sensores que presenten fallas.

1. Escriba el código en Pandas para filtrar y mostrar **únicamente** las filas donde el `estado` sea `.'Inactivo.'` **Y** (`&`) la `temperatura` sea superior a 30 grados.
2. **Análisis Crítico:** ¿Por qué en Pandas utilizamos el operador `&` en lugar de la palabra reservada `and` de Python tradicional al hacer filtrado múltiple?

‘Vectorizar no es solo una optimización de código, es cambiar la mentalidad algorítmica iterativa por una mentalidad algebraica.’